If Ruby comes across a line anywhere in the source containing just __END__ with no leading or trailing whitespace, it treats that line as the end of the program—any subsequent lines will not be treated as program code. Those subsequent lines are treated as data, and these lines can be read into the running program using the global IO object DATA.

```
DATA.each_line do |line|
  p line
end

__END__
line one
line two
```

produces:

```
"line one\n"
"line two\n"
```

BEGIN and END Blocks

Every Ruby source file can declare blocks of code to be run as the file is being loaded (the BEGIN blocks) and after the program has finished executing (the END blocks):

```
BEGIN {
  begin code
}

END {
  end code
}
```

A program may include multiple BEGIN and END blocks. BEGIN blocks are executed in the order they are encountered. END blocks are executed in reverse order.

These blocks are admittedly a little obscure, and seem mostly to be used for command line one-line commands to allow initialization or teardown around the actual one-line command.

```
$ ruby -ne "BEGIN { result = "" }; END { p result }; result << gets.upcase" testfile
```

With the -n acting as a while loop and the blocks, this code is equivalent to:

```
result = ""
while gets
  result << gets.upcase
end
p result
```

Because the gets is used as the while condition, this code prints every other line of the file.

The Basic Types

The basic types in Ruby are numbers, strings, arrays, hashes, ranges, symbols, and regular expressions. Although all types in Ruby are implemented as classes, these are the types that have special syntax for creation of literal values of these types. Lambda literals are covered in the next chapter, we'll cover the rest here.